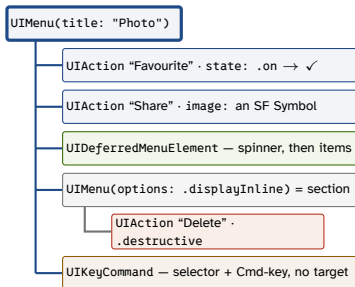
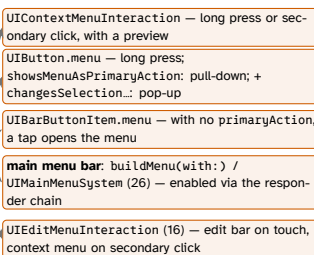


In one line: A menu is a **value tree** of `UIMenuElements` — `UIMenu` (a group) holding `UIAction` (a closure), `UICommand` (a *selector* sent up the **responder chain**) and `UIDeferredMenuElement` (loaded lazily). The **same tree** is shown by many presenters: a context-menu interaction (with a preview), a button, a bar item, the edit menu, the **main menu bar** (Mac; iPad since iPadOS 26).

one menu tree (Composite)



shown by



same UIMenu value everywhere

context menu, step by step

- 1 long press → `configurationForMenuAtLocation` (required)
 - 2 `highlightPreviewForItemWithIdentifier` → a `UITargetedPreview` lifts the view (`visiblePath` = its corners)
 - 3 `actionProvider(suggestedActions)` builds the tree; the `previewProvider` VC shows `willDisplayMenuFor`
 - 4a action tapped → its handler runs, menu closes
 - 4b preview tapped → `willPerformPreviewActionForMenuWithAnimator.addCompletion { push(detail) }`
 - 5 `willEndFor` + `dismissalPreviewForItemWithIdentifier` — the identifier finds the cell again
- empty configuration = lift, then cancel ("a menu lives here, nothing now");
nil = no feedback

How it works

- **UIAction:** title, subtitle, image, handler; attributes `.disabled`, `.destructive`, `.hidden`, `.keepsMenuPresented` (16); state `.on/.off/.mixed`. **UIMenu** options: `.displayInline`, `.singleSelection` (radio), `.displayAsPalette` (17); `preferredElementSize` `.small/.medium/.large` (16). Children are fixed: build a new menu and re-assign `button.menu`.
- **Deferred:** `UIDeferredMenuElement { completion in ... }` (14) is **cached** after the first load; `.uncached` (15) re-asks each time. iOS 26 `.usingFocus(identifier:shouldCacheItems:)`: the focused responder's `provider(for:)` fills it — per-window main-menu items.
- **Context menu:** add a `UIContextMenuInteraction(delegate:)` to the view, or in a list implement `contextMenuConfigurationForItemsAt:point: (16): []` = empty space, several paths = multi-selection. `previewProvider nil` = snapshot of the view.
- **Main menu:** override `buildMenu(with builder:)` in the app delegate (13): `builder.system == .main`, `insertChild(_:atStartOfMenu:)`, `remove(menu:)`; `UIMenuSystem.main.setNeedsRebuild()`. iPadOS 26 menu bar: `UIMainMenuSystem.shared.setBuildConfiguration(config) { builder in ... }`, once at launch; storyboard menu bars unsupported. iOS 18: commands reach iPhone apps too (iPhone Mirroring).
- **Validation = responder chain:** a `UICommand` has no target — UIKit walks up from the **first responder** to the first one whose `canPerformAction(_:withSender:)` is true; adjust title/state in `validate(_ command:)`. None → disabled.
- **Edit menu:** `UIEditMenuInteraction` (16) replaces `UIMenuController.presentEditMenu(with: UIEditMenuConfiguration(identifier:sourcePoint:))`; `delegate menuFor:suggestedActions:`. Text views: `editMenuForTextIn:suggestedActions:`.
- **Pointer** (iPadOS 13.4): `UIPointerInteraction` → `UIPointerStyle(effect: .highlight/.lift/.hover)`; buttons: `isPointerInteractionEnabled`. **Drag & drop** (11): `UIDragInteraction` returns `[UIDragItem]`; `UIDropInteraction: canHandle` → `UIDropProposal(.copy/.move)` → `performDrop`; drag is off by default on iPhone (unverified).

Example — cell menu with preview, pop-up sort button

```
func collectionView(_ cv: UICollectionView, contextMenuConfigurationForItemsAt
  ips: [IndexPath], point: CGPoint) -> UIContextMenuConfiguration? {
  guard ips.count == 1,
    let id = ds.itemIdentifier(for: ips[0]) else { return nil }
  return UIContextMenuConfiguration(identifier: id as NSString, // NSCopying
    previewProvider: { DetailVC(id: id) } { _ in
      let del = UIAction(title: "Delete", attributes: .destructive) {
        [weak self] _ in self?.delete(id)
      }
      let share = UIDeferredMenuElement.uncached { [weak self] done in
        Task { done(await self?.shareActions(id) ?? []) } // spinner first
      }
      return UIMenu(children: [share,
        UIMenu(options: .displayInline, children: [del])]) } } // a section
    sortBtn.menu = UIMenu(options: .singleSelection, children: sorts.map { s in
      UIAction(title: s.title, state: s == sort ? .off : .on : { _ in pick(s) } })
    }
    sortBtn.showsMenuAsPrimaryAction = true // pull-down ...
    sortBtn.changesSelectionAsPrimaryAction = true // ... + this = pop-up
```

A button with a menu — what the flags do

menu	showsMenu...	changes...	behaviour
nil	—	true	toggle: flips <code>isSelected</code>
set	false	false	tap = primary action, long press = menu
set	false	true	toggle + long-press menu
set	true	false	pull-down: menu on touch-down
set	true	true	pop-up: tracks selection, title follows

Traps

- ! “The button’s menu never shows” — without `showsMenuAsPrimaryAction` it is a *long-press* menu.
- ! `IndexPath` captured in a handler or used as the identifier — the diffable snapshot may move it; use the item **id**.
- ! The control retains the menu, the menu its actions: `self→button→menu→handler→self` is a cycle — `[weak self]`.
- ! Stale items in a deferred element: it is cached after the first load — `.uncached`.
- ! Menu-bar item always grey: nothing between the first responder and the app answers `canPerformAction` with true.

Remember

“One tree, many presenters; actions carry closures, commands ride the responder chain.” Context menu = `configuration` → `preview` → `menu` → `commit`.